toychain

Tamper-Evidence, Difficulty vs. Effort, and Design Rationale

Research Report

Backend Engineering Internship — Golang Developer Assessment

Name: Teshan G.H.P

Course / Module: Intern Assessment

Date: 14/07/2026

Overview

This report documents three experiments run against my own blockchain implementation: tamper-evidence, difficulty versus mining effort, and a design write-up covering the hashing scheme and validation logic. All results below come from actual runs of my own code.

1. Tamper-Evidence

1.1 Setup

I built a small chain for this experiment: a coinbase transaction minting 100 units to alice, and a transfer of 30 units from alice to bob, mined together into block 1.

1.2 Before Tampering — Validation Output

```
$ toychain -data demo.json validate
chain is VALID
```

```
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain> .\toychain.exe -data demo.json -difficulty 3 add-tx "-" alice 100
queued: - -> alice : 100
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain> .\toychain.exe -data demo.json add-tx alice bob 30
queued: alice -> bob : 30
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain> .\toychain.exe -data demo.json mine
mined block 1
  hash:     0007cf5500fb1aa3b2a1485217e2b8e3531f5e537a66e14d70b0e989766862fc
  nonce:    21529
  attempts: 21530
  elapsed:  31.9672ms
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain> .\toychain.exe -data demo.json validate
chain is VALID
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain>
```

Figure 1: Chain validation before tampering

1.3 The Tamper

I modified the following field directly in the persisted chain.json file, without re-mining:

Field	Original value	Tampered value
Transaction: alice → bob, amount	30	999999

1.4 After Tampering — Validation Output

```
$ toychain -data demo.json validate
chain is INVALID
  first bad block: 1
  reason: block 1: stored hash 000b7377bf16... does not match
  recomputed hash 082d24f18284... (block contents were altered
  after mining)
```

Figure 2: Chain validation after tampering

1.5 Why the Chain Becomes Invalid

Validate() performs two checks relevant to this tamper, applied in sequence for every block in the chain:

```
if b.CalculateHash() != b.Hash {
    // reason: stored hash does not match recomputed hash
}
```

```
if b.PrevHash != prev.Hash {
    // reason: prev-hash does not match previous block's hash
}
```

The transaction list is part of the input to CalculateHash(), so altering the amount field changes the hash the function produces. The block's stored Hash field, however, still holds the value computed at mining time, before the edit. Because SHA-256 exhibits the avalanche property — a single-bit change to the input produces a statistically unrelated output — there is no way to alter a transaction and coincidentally preserve the original hash; the probability of such a collision is on the order of 1 in 2^{256}, which is not a practical concern.

The first check, hash recomputation, is what actually flags this tamper, and it does so at block 1 itself, before validation even reaches block 2's prev-hash check. Had I instead re-mined block 1 after tampering — searching for a new nonce so its hash satisfies the difficulty target again — the first check would pass, but the second check would then fail at block 2, since block 2's PrevHash still references the original, now-superseded hash of block 1. I verify this second scenario directly in my test suite (TestValidate_DetectsBrokenPrevHashLink), which re-mines a tampered block and confirms the chain is still correctly reported as invalid, one block downstream.

2. Difficulty vs. Effort

2.1 Method

I implemented an experiment command directly in the CLI (toychain experiment 6) to make this measurement reproducible without any external tooling. It mines one independent block at each difficulty level from 1 through 6 and records the number of nonce attempts and the elapsed time.

2.2 Results

Difficulty	Attempts (nonces tried)	Time (ms)	Hash prefix
1	2	0.000	08ea4730e8…
2	385	0.658	00b8e8b7c0…
3	5,778	6.927	000d6f6b7d…
4	63,993	65.857	0000604e41…
5	97,500	83.905	00000d08ae…
6	3,683,870	3,101.529	000000ac94…

```
PS D:\PROJECTS\OFFICE\botcalm\4th week\toychain_all\toychain> .\toychain.exe experiment 6
Difficulty vs. effort

| Difficulty | Attempts (nonces tried) | Time (ms) | Hash prefix |
|---|---|---|---|
| 1 | 2 | 0.000 | `08ea4730e8...` |
| 2 | 385 | 0.658 | `00b8e8b7c0...` |
| 3 | 5778 | 6.927 | `000d6f6b7d...` |
| 4 | 63993 | 65.857 | `0000604e41...` |
| 5 | 97500 | 83.905 | `00000d08ae...` |
| 6 | 3683870 | 3101.529 | `000000ac94...` |

Interpretation: each extra required leading hex zero narrows the
target space by a factor of 16 (one hex digit = 4 bits = 2^4 values),
so attempts and time should grow roughly geometrically (~16x per
level), not linearly. Compare the ratio between consecutive rows
above to your own run's numbers.
```

Figure 3: Output of `toychain experiment 6`

2.3 Trend Analysis

The growth in effort is geometric rather than linear, but a single trial can deviate substantially from the theoretical average, and my own results illustrate this clearly. Each additional required leading hexadecimal zero narrows the space of acceptable hashes by a factor of 16, since one hexadecimal digit represents 4 bits and 2^4 equals 16. On average, this predicts roughly a 16x increase in attempts per level. My measured ratios were 192.5x from difficulty 1 to 2 (2 to 385 attempts), 15.0x from difficulty 2 to 3 (385 to 5,778) — almost exactly the theoretical figure — 11.1x from difficulty 3 to 4 (5,778 to 63,993), only 1.5x from difficulty 4 to 5 (63,993 to 97,500), and 37.8x from difficulty 5 to 6 (97,500 to 3,683,870).

The 192.5x first-level ratio is not evidence that difficulty 1 to 2 is unusually costly — it is the opposite: difficulty 1 finished in only 2 attempts, an exceptionally lucky result well below its own theoretical average of roughly 16, which inflates the ratio to whatever comes next regardless of how difficulty 2 behaves on its own terms. Similarly, the 1.5x ratio from difficulty 4 to 5 does not mean difficulty 5 was nearly as easy as difficulty 4 in general — it means this specific difficulty-5 search happened to succeed unusually early. Both are consistent with mining being a memoryless random search: each attempt has a roughly 1-in-16 chance of meeting one additional leading zero, independent of how many attempts came before, so any individual trial's attempt count is drawn from a wide distribution around the theoretical mean rather than sitting close to it.

The middle of the table shows this converging toward theory more closely: difficulty 2 to 3 landed at 15.0x, within a few percent of the theoretical 16x. This suggests that with more trials — or by averaging several runs at each difficulty level rather than reading off a single one — the overall pattern would converge much more tightly on the expected 16x geometric growth than any single run does in isolation.

From a practical standpoint, difficulty 3 or 4 completes effectively instantaneously on standard laptop hardware, comfortably within the assignment's requirement that a reviewer should not wait more than a few seconds for a mined block. Difficulty 6 took just over 3 seconds in this run, though I observed it take considerably longer —

upward of 20 seconds — in other trials, another reflection of how variable a single proof-of-work search can be. On this basis, I selected difficulty 3 as the default: fast enough for interactive use, while still requiring a non-trivial, measurable amount of computation.

3. Design Write-Up

3.1 Hashing Scheme

A block's hash is computed as SHA-256 over five fields, concatenated in a fixed order with "|" separators. The Hash field itself is deliberately excluded from this calculation, since including it would be circular — the hash cannot be an input to its own computation.

Order	Field	Why it's included
1	Height	Makes the block's position part of its identity.
2	Timestamp	Ties the hash to the moment the block was created.
3	Transactions	Included in full, so any change to a transfer changes the hash.
4	PrevHash	Explicitly links this block to the one before it.
5	Nonce	Placed last, since mining works by searching over this value only.

These fields are concatenated explicitly rather than serialized via json.Marshal on the whole struct. This adds a small amount of code but removes any ambiguity about what feeds the hash and in what order — the scheme can be documented and verified directly, independent of how Go's JSON encoder happens to behave. Hashing the same block twice always produces the same result, because every input is either a primitive value or a struct with a fixed field order; the payload contains no map, which is the one common Go data structure whose iteration order is not guaranteed.

3.2 Validation and Chain-Wide Integrity

Validate() walks the chain from genesis to tip and checks four properties on every block, stopping at the first failure:

- Recomputed hash matches the stored hash — detects direct modification of a block's data.
- PrevHash matches the previous block's actual hash — detects tampering even when the altered block has been re-mined to satisfy the first check again.
- The hash satisfies the proof-of-work target — confirms genuine computational work was performed for that block.
- Height and timestamp remain consistent with the previous block.

The first two checks carry the core integrity guarantee. Undetectably altering block k requires re-mining it so that its hash again satisfies the difficulty target, and then repeating that process for every subsequent block, since each one's PrevHash field references the original hash of its predecessor. Modifying PrevHash changes that block's own hash, which in turn invalidates the next block's link, propagating forward to the tip. This is demonstrated directly in the test suite: one test tampers with a block and re-mines it, and the chain is still correctly reported as invalid, because the following block still references the pre-tamper hash.

3.3 Limitations

The guarantees described above hold only under the assumptions built into this implementation, and it is worth being explicit about what they do not cover:

- No defense against a majority-hashpower rebuild. Nothing in this implementation prevents an attacker with sufficient computational resources from re-mining a tampered block and every block after it. In a real network, this is deterred by the cost of outpacing honest miners; in this single-process toy, that deterrent does not exist, since there is no competing network to outpace.
- No replay protection. Transactions carry no signature and no per-account sequence number, so nothing distinguishes a legitimate resubmission from a malicious replay of a previously valid transaction. A production system would need to close this gap before signatures alone would be sufficient.
- No fork detection or resolution. Because there is only one process and one copy of the chain, there is no mechanism to detect or resolve a scenario where two valid-looking chains diverge — a core concern in any system with multiple independent participants.

4. Discussion Questions

4.1 Why does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in this local implementation?

Tampering is trivial in this implementation because it runs as a single, isolated process. There is no independent copy of the chain to disagree with, and no concurrent mining activity taking place while a modification is made. In a real network, altering an old block requires re-mining that block and every block after it — and this is not a fixed, self-paced amount of work. While an attacker re-mines the affected blocks, honest participants continue extending the legitimate chain, so the attacker is not working toward a fixed target but a continuously advancing one. As long as honest miners collectively control a majority of the network's hashing power, they extend the legitimate chain faster than an attacker can catch up and rewrite history. Even if an attacker did complete an alternate chain, every other participant independently validates incoming chains and would reject a version that does not match the one they have already accepted.

4.2 Proof-of-work is one way to decide who adds the next block. Name at least one alternative and give one advantage and one drawback versus proof-of-work.

Proof-of-Stake is the most direct alternative. Rather than requiring computational work, validators lock up currency as collateral, and the protocol selects who proposes the next block roughly in proportion to the size of their stake; validators who act dishonestly forfeit that stake. The principal advantage is energy efficiency: there is no continuous, large-scale hashing competition running across the network, which is the most frequently cited criticism of proof-of-work systems. The principal drawback is a tendency toward concentration of influence — larger stakes translate into more frequent selection, which can compound advantages for participants who already hold the most. A related, more technical drawback is the "nothing-at-stake" problem: because proposing a block under Proof-of-Stake costs little beyond a signature, a validator has comparatively little disincentive to support multiple competing chains simultaneously during a fork, unlike under proof-of-work, where computational effort must be split between competing chains and cannot be duplicated for free.

4.3 List three concrete ways your toy differs from a production blockchain. Pick one and sketch how you would add it.

- No peer consensus — this implementation runs as a single process with one authoritative copy of the chain. A production network must reconcile many independent copies and resolve disagreements between them.

- No digital signatures — a transaction can claim to originate from any account, since nothing in the current implementation verifies that the sender authorized it.
- No Merkle tree — a block's transaction list is hashed as a single flat JSON array rather than summarized as a tree with one root hash, meaning verification that a specific transaction was included requires the entire block rather than a short inclusion proof.

Of these three, digital signatures address the most significant practical gap, since the current ledger has no mechanism to prevent an account from being debited without its owner's authorization. A concrete implementation would proceed as follows: generate an Ed25519 keypair for each account; add a Signature field to the Transaction struct; require the sender to sign a canonical serialization of the transaction's fields (sender, recipient, amount) with their private key before the transaction is accepted into the mempool; and extend Balances.Validate() to verify that signature against the sender's known public key, in addition to the balance check it already performs.

This alone would not be sufficient, however. Without a per-account sequence number attached to each transaction, a previously valid, correctly signed transaction could be resubmitted and accepted a second time — a replay attack. Closing this gap would require each transaction to carry an incrementing nonce scoped to the sender's account, with Validate() rejecting any transaction whose nonce does not match the sender's next expected value.

Sources

[TODO — list anything you referenced beyond your own code and experiments: Go standard library docs, articles on proof-of-work or Merkle trees, etc. Doesn't need to be formal citation format, just honest about what you read.]